

Calculus 1-3 for Machine Learning

A Visual & Code-First Mathematical Primer

Calculus is the mathematical engine of machine learning. It provides the framework for modeling how adjustments to model parameters change the output error. This guide builds a bottom-up, visual, and code-connected overview of Calculus 1, 2, and 3, illustrating their direct application to optimization and neural network backpropagation.

1. Calculus 1: Limits, Derivatives, and Optimization

At the center of Calculus 1 is the **derivative**, which defines the instantaneous rate of change of a function. It is defined mathematically as the limit of a secant line's slope as the interval approaches zero: $\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. Geometrically, this represents the slope of the tangent line at a specific point (see Figure 1 below).

In training machine learning models, derivatives tell us whether to increase or decrease model parameters to decrease the loss function. The derivative of a composite function is calculated using the **Chain Rule**: $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$, which is the foundation for propagating gradients backward through layers of a neural network.

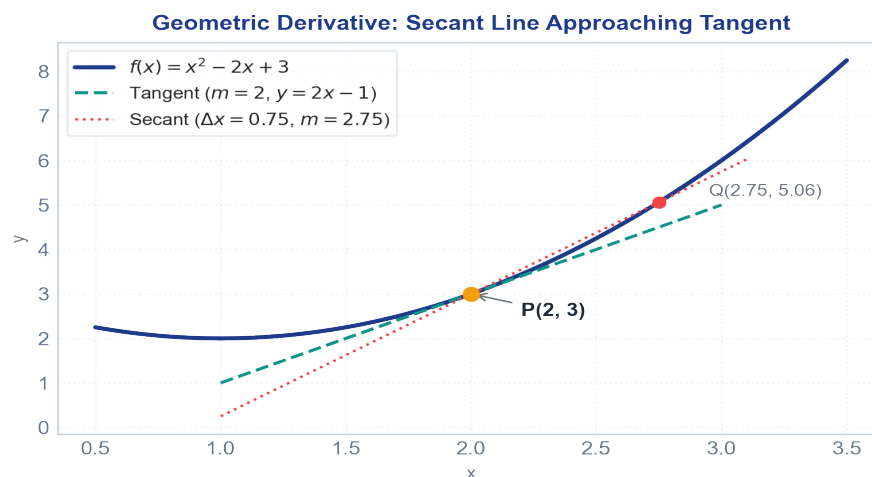


Figure 1: Tangent line as limit of secant line (derivative definition).

2. Calculus 2: Integration and Series Approximation

Integration is the inverse operation of differentiation. The definite integral measures the accumulated area under a curve between two points (see Figure 2). In ML, integration is key to computing probabilities, expected values, and continuous distributions (e.g. density estimation). While analytic integration can be difficult, we approximate integrals numerically using discrete sums (Riemann sums) — a core technique in computational math.

Another crucial concept in Calculus 2 is the **Taylor Series**: $f(x) \approx \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$, which allows us to approximate a complex, non-linear function locally near a point using a polynomial. In optimization, first and second-order Taylor approximations form the basis of gradient descent and Newton-Raphson solvers.

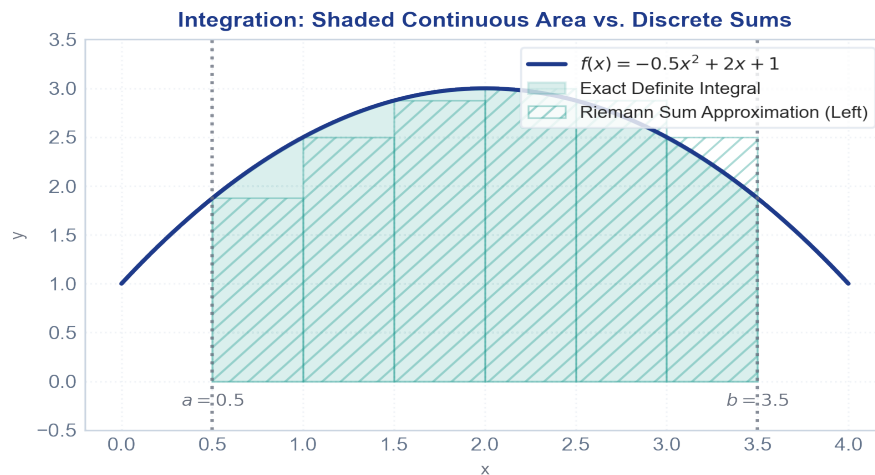


Figure 2: Numerical Riemann approximation of a continuous definite integral.

3. Calculus 3: Multivariable Calculus, Gradients, and Curvature

Machine learning models rarely deal with single variables; inputs and weights live in high-dimensional spaces. Multivariable calculus extends single-variable concepts to functions of several variables. For a multivariable scalar function, the **Gradient** is a vector containing its partial derivatives: $\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$. Geometrically, the gradient vector points in the direction of steepest ascent. Minimization algorithms (like Gradient Descent) update weights by moving in the negative gradient direction to reach local minima (see Figure 3).

For vector-valued multivariable functions, the matrix of first partial derivatives is the **Jacobian** ($J = \left[\frac{\partial f_i}{\partial x_j} \right]$). For scalar multivariable functions, the matrix of second partial derivatives is the **Hessian** ($H = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]$), which represents the local curvature of the surface and determines whether a stationary point is a minimum, maximum, or saddle point.

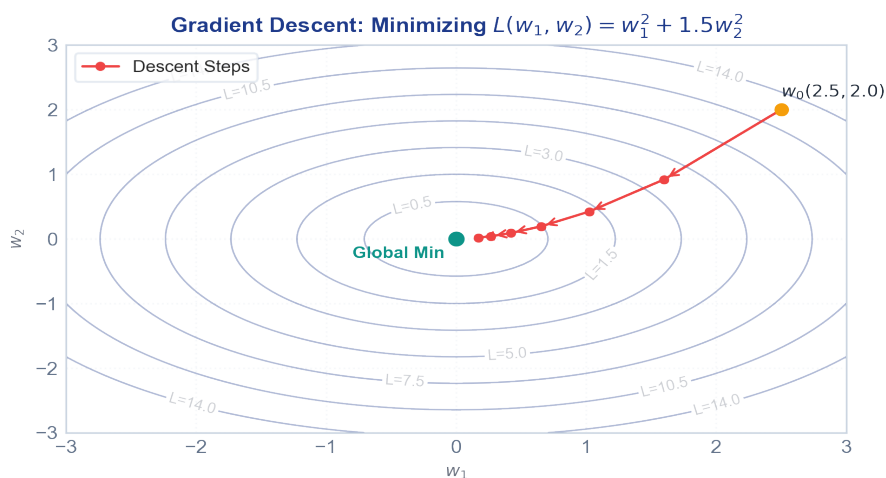


Figure 3: Gradient descent steps moving down levels of a loss contour surface.

4. Machine Learning Integration: Neural Network Backpropagation

The backpropagation algorithm trains a neural network by computing the gradient of the loss function L with respect to each weight w_{ij} . Using the multivariable chain rule, this is written as:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

This formula decomposes the derivative into three parts: (1) how the loss changes with the node activation, (2) how the activation changes with the net input, and (3) how the net input changes with the weight. Below is a simplified Python implementation of a forward and backward pass for a single neuron:

```
import numpy as np

def forward_backward_neuron(x, w, b, y_true, eta=0.1):
    # Forward pass
    z = np.dot(x, w) + b
    a = 1 / (1 + np.exp(-z)) # Sigmoid activation
    loss = 0.5 * (a - y_true)**2 # MSE loss

    # Backward pass (Calculus Chain Rule)
    dLoss_da = a - y_true
    da_dz = a * (1 - a) # Derivative of sigmoid
    dz_dw = x
    dz_db = 1.0

    dLoss_dw = dLoss_da * da_dz * dz_dw
    dLoss_db = dLoss_da * da_dz * dz_db

    # Parameter update
    w -= eta * dLoss_dw
    b -= eta * dLoss_db
    return loss, w, b
```

ML INSIGHT: Gradient descent relies entirely on Calculus 3. In deep neural networks, millions of weights are updated simultaneously by computing partial derivatives using the generalized chain rule. Understanding the math behind backprop prevents treating neural networks as black boxes.

Calculus 1-3 Quick Reference Sheet

Key formulas, derivatives, and multivariable rules for machine learning.

Topic / Formula Name	Mathematical Definition	ML Application / Context
Limit Definition of Derivative	$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$	Formalizes local instantaneous change and slopes.
Chain Rule (1D)	$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$	Allows differentiating composite functions; backprop foundation.
Gradient Vector	$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$	Vector of partial derivatives; points to steepest ascent.
Jacobian Matrix	$J = \left[\frac{\partial f_i}{\partial x_j} \right]$	First-order partial derivatives for vector outputs; dimensions alignment.
Hessian Matrix	$H = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]$	Second-order partial derivatives; represents curvature and saddle points.
Taylor Series Approximation	$f(x) \approx \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$	Locally linearizes or quadraticizes a non-linear loss function.
Derivative Rules	$d/dx(x^n) = n \cdot x^{(n-1)}$ $d/dx(e^x) = e^x$ $d/dx(\ln x) = 1/x$	Standard rules for computing weight gradient updates.
Gradient Descent Step	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$	Weight parameter update rule in optimization.